Lomads

Soulbound Token

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

28 September 2023

Introduction **3**

About Lomads 3

About ImmuneBytes 3

Project Overview **4**

Documentation Details **5**

Audit Goals **5**

Audit Process & Methodology **6**

Audit Details **7**

Security Level References **8**

Severity Status References **9**

Severity Status **9**

Findings **10**

Critical Severity Issues **11**

High Severity Issues **11**

Medium severity issues **11**

Low severity issues **16**

Informational **17**

Gas Optimization **19**

Concluding Remarks **20**

Disclaimer **20**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Lomads

Lomads is the "CommandCentral" for decentralized collectives, revolutionizing efficiency in burgeoning sectors like Decentralized Science (DeSci) and Open Source. They're powering the future of collaborative economies by providing an all-in-one platform for real-time validation, fund distribution, and seamless tool integration.

Visit https://www.lomads.xyz/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Project Overview

The provided Solidity smart contract is a specialized implementation of an ERC-721 token, designed for creating non-fungible tokens (NFTs) on the Ethereum blockchain. The contract goes beyond the basic ERC-721 standard by incorporating additional features such as meta-transactions, URI storage for tokens, and unique payment and minting options.

Key Features

- **Custom Minting:** The contract allows for the minting of new tokens through a safeMint function. This function not only creates a new token but also associates it with a unique URI and requires a specific set of conditions to be met for successful minting.
- **Meta-Transaction Support:** The contract is equipped to handle meta-transactions, enabling users to interact with the contract without the need for Ether in their accounts. This is achieved through the implementation of the ERC2771 standard.
- **URI Storage:** The contract uses ERC721URIStorage to associate a unique URI with each minted token. This allows for the storage of metadata off-chain while maintaining a link to it on-chain.
- **Ownership Management:** Leveraging the Ownable pattern, the contract allows for basic authorization control, enabling certain functions to be restricted to the contract owner.
- **Minting Price and Payment:** The contract has a feature to set a price for minting new tokens. It also allows for the specification of a payment method, providing flexibility in how users can pay for minting.
- **Whitelisting Option:** The contract includes an optional feature that requires users to be whitelisted before they can interact with it, providing an additional layer of control.
- **Token Burn and Revoke:** The contract allows token holders to burn their tokens, effectively removing them from circulation. Additionally, the contract owner has the ability to revoke tokens, offering a way to manage tokens post-minting.
- **Immutable Signer:** The contract specifies an immutable signer address that is authorized to sign messages for secure minting.
- **Replay Attack Prevention:** The contract employs a unique id parameter and an internal counter to ensure that each minting operation is unique, preventing replay attacks.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. https://lomads-1.gitbook.io/lomads/

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

The ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors, which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

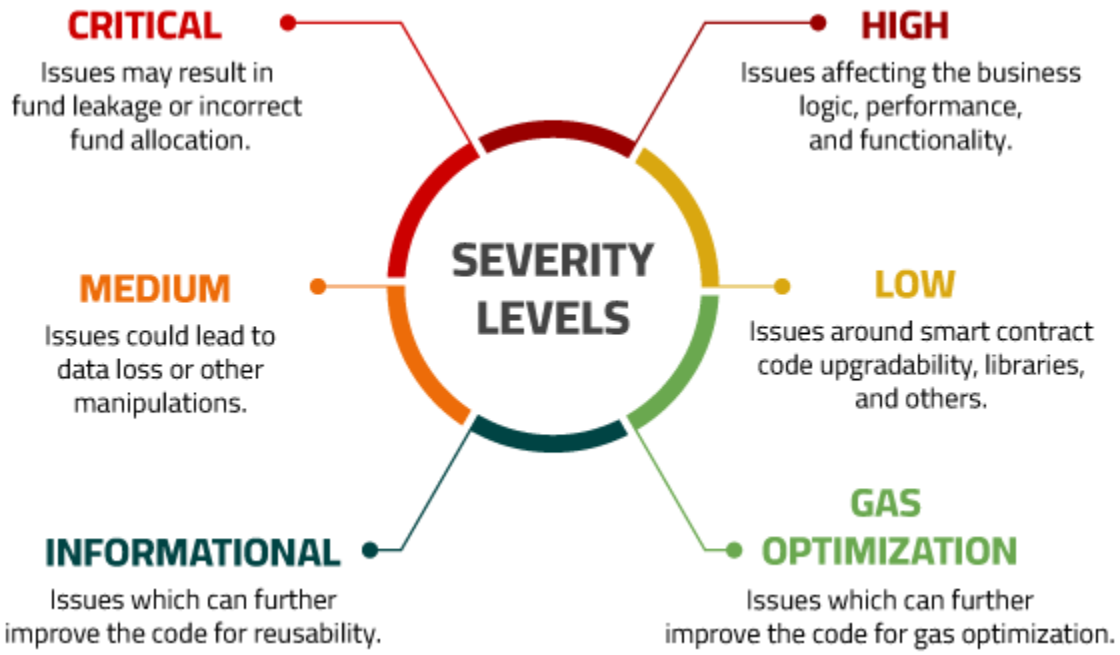
Audit Details

Project Name	Lomads
Platform	Polygon
Contract	Code Link | Soulbound Token - Lomads
Languages	Solidity
Auditors	Kaif Ahmed | Gul Hameed
Report Type	Final

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	-	3	1	1	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	-	-	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Findings

ID	Findings	Severity	Status
SBT - 001	Missing Input Validation in Various Functions	**Medium**	**Fixed**
SBT - 002	Unrestricted Token Revocation by Admin	**Medium**	**Fixed**
SBT - 003	Missing Event Emission in setProperties Function	**Medium**	**Fixed**
SBT - 004	Hardcoded signerAddress in Smart Contract	**Low**	**Fixed**
SBT - 005	Code Organization and Layout	**Informational**	**Fixed**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

No issues were found.

Medium severity issues

SBT - 001	Missing Input Validation in Various Functions		
Function Name	**safeMint()** **setProperties()** **Burn()** **revoke()**		
Rating	Severity: Medium	Likelihood: High	Impact: Low

Description

The contract lacks sufficient input validation in several functions. While some checks are in place, such as signature validation in the safeMint function, other important validations are missing. This could lead to unintended behavior, including the setting of invalid state variables or unauthorized actions, thereby compromising the contract's integrity and security.

Code Affected

safeMint Function: Missing validation for uri and transaction_type.

```
function safeMint(
    string memory uri,
    uint256 id,
    string memory payment,
    bytes memory signature,
    string memory transaction_type
) external {
    // ... (code)
}
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Impact An empty or invalid URI could be set, and transaction_type is not validated, which could lead to unexpected behavior.

setProperties Function: Missing validation for treasury, _mintToken, and _priceInWei.

```
function setProperties(
    address treasury,
    address _mintToken,
    uint256 _priceInWei,
    bool choice
) external onlyOwner {
    // ... (code)
}
```

Impact Critical addresses could be set to the zero address, and _priceInWei is not validated, potentially allowing for free minting.

burn Function: Missing validation for tokenId.

```
function burn(uint256 tokenId) external {
    // ... (code)
}
```

Impact Tokens that don't exist could potentially be targeted, leading to function failure or unexpected behavior.

revoke Function: Missing validation for tokenId.

```
function revoke(uint256 tokenId) external onlyOwner {
    // ... (code)
}
```

Impact The contract owner could attempt to revoke tokens that don't exist, leading to function failure or unexpected behavior.

Recommendation

- safeMint Function: Add checks to validate the uri and transaction_type parameters.
- setProperties Function: Add checks to ensure that the address parameters (treasury, _mintToken) are not set to the zero address. Optionally, validate _priceInWei.
- burn and revoke Functions: Add checks to validate that the tokenId exists in the contract.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

SBT - 002	Unrestricted Token Revocation by Admin		
Function Name	**revoke()**		
Rating	Severity: Medium	Likelihood: High	Impact: Low

Description

The revoke function in the contract allows the contract owner (admin) to unilaterally burn any token by specifying its tokenId. While this provides a high level of control to the admin, it raises several concerns related to centralization, user trust, and potential abuse. Depending on the intended use case and the level of trust users have in the admin, this feature could be problematic.

Code affected

safeMint Function: Missing validation for uri and transaction_type.

```
function revoke(uint256 tokenId) external onlyOwner {
    _burn(tokenId);
  }
```

Impact

- Centralization: This feature centralizes control over the tokens, potentially undermining the decentralized nature of blockchain systems.
- User Trust: Trust in the system could be eroded if users know their tokens can be arbitrarily burned by the admin.
- Regulatory Risks: Depending on jurisdiction and token nature, this control could attract regulatory scrutiny.
- Market Impact: If tokens have financial value, arbitrary burning could disrupt the market.
- Transparency and Abuse: Lack of transparent criteria for revocation could lead to abuse.

Recommendation

- Remove the Function: If it's not essential for the contract's intended use case, consider removing it to enhance decentralization and trust.
- Add Conditions: If revocation is necessary, add transparent and agreed-upon conditions under which a token can be revoked.
- Event Logging: Emit an event when a token is revoked to maintain transparency.
- Multi-Signature Requirement: Consider requiring multiple signatories to execute a burn, reducing the risk of unilateral actions.
- User Notification: Implement a mechanism to notify users if their token is at risk of being revoked.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

SBT - 003	Missing Event Emission in setProperties Function		
Function Name	**setProperties()**		
Rating	Severity: Medium	Likelihood: High	Impact: Low

Description

The setProperties function allows the contract owner to update various important state variables such as mintPrice, mintToken, treasuryAddress, and mustBeWhitelisted. However, the function currently does not emit any events to log these changes. The absence of event logging makes it difficult to track changes, which could be crucial for auditing and transparency.

Code affected

safeMint Function: Missing validation for uri and transaction_type.

```
function setProperties(
  address treasury,
  address _mintToken,
  uint256 _priceInWei,
  bool choice
) external onlyOwner {
  // ... (existing code)
}
```

Impact

- Lack of Transparency: Without event logging, it's challenging to track when and by whom these important state variables were changed.
- Auditability: The absence of events makes it more difficult to audit the contract's behavior over time.
- User Trust: Users may have less trust in a system where significant changes can occur without transparent logging.

Recommendation

Add an event that gets emitted whenever the setProperties function is called. This will provide a transparent and auditable trail of all changes made to these critical state variables.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Fixed Code

```
// Add an event to log changes
event PropertiesSet(
    address newTreasury,
    address newMintToken,
    uint256 newPriceInWei,
    bool newMustBeWhitelisted,
    address by
);

function setProperties(
    address treasury,
    address _mintToken,
    uint256 _priceInWei,
    bool choice
) external onlyOwner {
    // ... (existing code)
    emit PropertiesSet(treasury, _mintToken, _priceInWei, choice, msg.sender);
}
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

SBT - 004	Hardcoded signerAddress in Smart Contract		
Function Name	signerAddress		
Rating	Severity: low	Likelihood: Medium	Impact: Low

Description

The signerAddress is hardcoded into the smart contract, which poses potential security and operational risks. While this approach is straightforward and transparent, it lacks flexibility and could become a single point of failure. If the private key for the hardcoded address is compromised, or if there is a need to change the signerAddress for operational reasons, the entire contract would be affected.

Code affected

```
address private signerAddress = 0x5Af6E42E155137907f1dEcbC957730Da2Dbf9741;
```

Recommendation

Constructor Parameter: Make the signerAddress a parameter in the constructor. This allows for flexibility in setting the address at the time of contract deployment.

```
constructor(
  // ... (other parameters)
  address _signerAddress
  // ... (other parameters)) {
  signerAddress = _signerAddress; }
```

Owner-Settable: Create a function that allows the contract owner to update the signerAddress. This function should emit an event for transparency. Note that this approach introduces a level of centralization and trust in the contract owner.

```
event SignerAddressUpdated(address newSignerAddress);
```

```
function updateSignerAddress(address newSignerAddress) external onlyOwner {
  signerAddress = newSignerAddress;
  emit SignerAddressUpdated(newSignerAddress); )
```

Multi-Signature Scheme: For added security, consider implementing a multi-signature scheme where multiple addresses are required to sign a message. This would reduce the risk associated with a single compromised key.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

SBT - 005	Code Organization and Layout		
Function Name	-		
Rating	Severity: Info	Likelihood: low	Impact: Low

Description

The current layout of the contract could be improved for better readability and maintainability. While the contract's functionality is not affected, adhering to best practices for code organization can make it easier for developers to understand and work on the contract.

Code affected

State Variables: Currently grouped but could be ordered by visibility.

```
Counters.Counter private _tokenIdCounter;
uint256 public mintPrice;
address public mintToken;
address payable public treasuryAddress;
address private signerAddress = 0x5Af6E42E155137907f1dEcbC957730Da2Dbf9741;
bool public mustBeWhitelisted;
```

Events: Properly placed after state variables.

```
event Attest(address indexed to, uint256 indexed tokenId);
event Revoke(address indexed to, uint256 indexed tokenId);
```

Constructor: Properly placed but could be grouped more closely with other special functions like fallback/receive functions if they are added later.

```
constructor(
    // ... (parameters)
) {
    // ... (code)
}
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Public Functions: These are mostly in place but could be grouped together more tightly.

```
function safeMint(
    // ... (parameters)
) external {
    // ... (code)
}
function getCurrentTokenId() public view returns (uint256) {
    // ... (code)
}
function setProperties(
    // ... (parameters)
) external onlyOwner {
    // ... (code)
}
function burn(uint256 tokenId) external {
    // ... (code)
}
function revoke(uint256 tokenId) external onlyOwner {
    // ... (code)
}
```

Internal Functions: These should be grouped together after the public functions.

```
function _check(
    // ... (parameters)
) public view returns (address) {
    // ... (code)
}
function isSafeMintMeta(string memory str1) internal pure returns (bool) {
    // ... (code)
}
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Overrides: These are fine at the end but could be grouped more clearly.

```
function _msgSender()
  // ... (code)
function _msgData()
  // ... (code)
function _burn(
  // ... (code)
function tokenURI(
  // ... (code)
function supportsInterface(
  // ... (code)
```

Recommendation

- Group state variables together at the top, ordered by visibility (public, internal, private).
- Place events right after state variables.
- If any modifiers are added later, place them after the events.
- Group public functions together.
- Group internal functions together, following the public functions.
- If any private functions are added later, place them last before the overrides.
- By following this recommended layout, the contract will be easier to read and maintain.

Gas Optimization

No issues were found.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of SBT, it was observed that the system contains Medium, Low and info severity issues. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Lomads platform or its product, nor is this audit investment advice.

Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS